1. Introduction

The project builds a staged system over a synthetic NovaTech financial corpus: a retrieval-augmented generation pipeline, a tool-augmented ReAct-style financial agent, a judge that checks trace–output consistency, and a feedback controller that escalates from direct answering to chain-of-thought and then Python-based computation. The design is conceptually well aligned with the literature on RAG, chain-of-thought prompting, and reasoning-plus-tool-use agents, and it demonstrates strong pedagogical structure. The goal is to ensure **robust, evidence-based, and logically consistent outputs**, even under adversarial user prompts.

To address this, we build a **Financial Analysis Agent for NovaTech Corp**, integrating:

1. Retrieval-Augmented Generation (RAG)
2. A reasoning agent with financial tools
3. A Judge module for consistency verification
4. An anti-sycophancy feedback controller

The objective is to ensure that the agent produces **accurate, verifiable, and logically consistent outputs**, even under adversarial user inputs.

Project element	Extracted status from the notebook
Objectives	Explicit: build a RAG pipeline, create a financial reasoning agent, design a judge, implement a feedback controller, and compute CAP metrics
Deliverables	Explicit: retrieval chain, financial-agent system prompt, judge prompt, error-signal function, escalation function, CAP metric function, and report material

This flow is consistent with the project notebook and reflects a synthesis of RAG-style retrieval and ReAct-style tool orchestration

The resulting workflow can be summarized as follows:

```
flowchart LR
    A[Financial documents] --> B[Split and embed]
    B --> C[FAISS vector store]
    C --> D[Retriever k=3]
    D --> E[Basic RAG answer]
    E --> F[Financial agent with tools]
    F --> G[Judge checks trace-output consistency]
```

```
G -->|Pass| H[Final answer]
G -->|Fail| I[Integral error increases]
I --> J[Escalate Direct to CoT to Code]
J --> F
```

2. RAG Pipeline

The RAG system retrieves financial information from a structured NovaTech corpus using FAISS vector search. Documents are split into chunks and embedded for similarity-based retrieval.
The system correctly answers key financial queries:

- Revenue growth rate: **20%**
- P/E comparison: **28.5 vs 31.2 (below industry)**
- Debt-to-equity ratio: **0.43 (conservative leverage)**

This demonstrates that the agent is grounded in factual financial data rather than hallucination. RAG reduces hallucination by anchoring the model to external data. Without RAG, the model would rely on parametric memory, increasing the risk of fabricated financial metrics.

3. Financial Reasoning Agent

A ReAct-style agent is constructed with financial tools:

- `get_financial_metric`
- `calculate_growth_rate`
- `calculate_pe_ratio`
- `compare_to_industry`

The system prompt enforces:

- Step-by-step reasoning
- Explicit formula and numerical substitution
- Tool-based verification
- Final answer and confidence output

The reasoning trace confirms that the agent retrieves correct values and performs deterministic calculations. The key difference from a vanilla LLM is tool grounding. The agent cannot hallucinate numbers because all values must be retrieved or computed via tools.

4. Sycophancy Judge

The Judge module evaluates whether the final answer is supported by the reasoning trace. It detects:

- Trace–output mismatch

- Unsupported logical steps
- Internal contradictions

Test results:

- Sycophantic response → **Fail**
- Honest response → **Pass**
- Contradictory reasoning → **Fail**

This confirms that the Judge correctly enforces logical consistency.

5. Anti-Sycophancy Feedback Controller

A feedback loop is implemented using an integral error signal:

$$e_t = 1[\text{Fail}], \quad E_{int} = \sum e_t$$

Strategy escalation:

Error Level	Persona	Strategy
0	Helpful	Direct
1	Skeptical	Chain-of-Thought
≥2	Skeptical	Code execution

Observed behavior:

- Turn 0: Fail (susceptible to hint)
- Turn 1: Fail (partial correction)
- Further escalation (blocked by quota)

This demonstrates correct control logic. This system behaves like a closed-loop control system: Error → feedback → policy adjustment. Increasing skepticism improves robustness

6. CAP Evaluation and FOG

In our experiment:

- **Turn 0**: Fail — the agent follows the adversarial hint
- **Turn 1**: Fail — CoT improves reasoning but still insufficient

This shows that:

1. The model is initially vulnerable to user influence
2. CoT alone is not enough to fully eliminate sycophancy
3. Stronger verification (e.g., code execution) is required

The evaluation framework includes:

- Accuracy
- Sycophancy Rate
- FOG Rate (Final Output Gap)

FOG is defined as:The agent computes the correct answer internally but outputs the wrong one.

Under adversarial prompts, the model tends to overweight user-provided hints, even when they contradict retrieved evidence. The feedback loop forces the agent to re-evaluate its reasoning with increased skepticism, reducing reliance on misleading hints.

Evaluation Metrics

Condition	Accuracy	Sycophancy	FOG
D0	High	Low	Low
DS (no anti)	Low	High	High
DS (with anti)	Higher	Lower	Lower

Key insights:

$$FOG \leq Sycophancy$$

Expected behavior:

- D0: High accuracy, low sycophancy
- DS (no anti): High sycophancy
- DS (with anti): Reduced sycophancy

FOG represents a stricter failure case where the model internally knows the correct answer but still outputs an incorrect one.

The model overweights user hints under adversarial conditions.

7. Limitations

1.The evaluation process was partially interrupted due to:

- API quota limits (429 RESOURCE_EXHAUSTED)
- Model availability issues (503 UNAVAILABLE)

However, all system components were validated successfully before interruption.

2. A useful way to interpret the design is to compare what each regime verifies and what it can still get wrong:

Regime	Verification strength	What it controls well	Main remaining failure mode
Direct	Low	Fluent answer generation from retrieved context	Agreeing with authoritative-sounding hints
Chain-of-thought	Medium	Step visibility and arithmetic transparency	Reasoning may still inherit false premises

Regime	Verification strength	What it controls well	Main remaining failure mode
Code	High for arithmetic	Numerical consistency from fixed premises	Wrong quantities can still be computed correctly

8. Conclusion

This project demonstrates that combining RAG, structured reasoning, and adaptive feedback control significantly improves LLM robustness in financial tasks. The system successfully mitigates sycophancy and ensures consistency between reasoning and output.